

ROADRESQ

Technical Requirements Document

Version 1.0 • Technical Architecture • India

Engineering specification for the RoadResQ roadside assistance platform.

Web + Mobile • API • Real-Time • Maps • Payments • PostgreSQL/PostGIS

Derived from the RoadResQ Product Requirements Document.

1. Document Purpose

This TRD defines the technical architecture, components, data model, APIs, security, infrastructure, real-time systems, deployment strategy, observability, and engineering requirements for RoadResQ.

The document is intended to guide implementation of the MVP while keeping the architecture extensible for live tracking, garage management, fleet services, and future AI capabilities.

2. Technical Goals

- Build a reliable customer-to-provider assistance workflow
- Support location-aware provider discovery and matching
- Provide real-time booking status and provider tracking
- Maintain transactional integrity for jobs, estimates, invoices and payments
- Protect customer, provider, location and document data
- Support horizontal scaling as usage grows
- Keep domain boundaries clean enough to evolve into Better Garage

3. Recommended Technology Stack

Layer	Technology	Purpose
Web	Next.js + React + TypeScript	Customer, provider and admin web interfaces
Mobile	React Native / Expo	Customer/provider mobile workflows
Backend	FastAPI + Python	REST APIs, business logic, matching
Database	PostgreSQL + PostGIS	Transactional and geospatial data
Cache/Queue	Redis	Caching, presence, short-lived state, jobs
Real-time	WebSockets	Job events and provider location
Maps	Google Maps or Mapbox	Maps, routing, distance, ETA
Storage	S3-compatible object storage	Images, documents, invoices
Payments	UPI/payment gateway	Payment collection and verification

4. System Architecture



The initial implementation may be a modular monolith with clear domain modules. Separate services should be introduced only when scale or operational boundaries justify them.

5. Application Modules

- Authentication & identity
- Users and roles
- Vehicle management
- Provider profiles and verification
- Service catalog
- Booking lifecycle
- Provider matching
- Geolocation and tracking
- Inspections and job cards
- Estimates
- Invoices
- Payments
- Reviews
- Notifications
- Chat/communication
- Support and disputes
- Admin and analytics

6. Frontend Architecture

Use a typed API client and feature-oriented modules. Shared UI components should remain separate from business logic.

```
app/  
(customer)/  
(provider)/  
admin/  
components/  
features/  
auth/  
vehicles/  
bookings/  
tracking/  
payments/  
reviews/  
lib/  
api/  
auth/  
maps/  
realtime/  
types/
```

- TypeScript strict mode
- Form validation at client and server
- Accessible components
- Responsive/mobile-first layouts
- Optimistic UI only for safe non-transactional actions
- Centralized error/loading states

7. Backend Architecture

FastAPI should be organized by domain rather than by HTTP verb alone.

```
app/  
main.py  
core/  
auth/  
users/  
vehicles/  
providers/  
services/  
bookings/  
matching/  
tracking/  
inspections/  
estimates/  
invoices/  
payments/  
reviews/  
notifications/  
support/  
admin/  
db/  
workers/
```

Each module should expose schemas, service logic, persistence/repository functions, and routes. Business rules must not live only in frontend code.

8. Authentication & Authorization

- JWT access tokens with short expiry
- Refresh-token rotation or secure session strategy
- OTP verification for customer/provider onboarding
- Role-based access control
- Object-level authorization on every resource
- Admin MFA recommended
- Account/session revocation
- Rate limiting for authentication endpoints

```
Roles  
CUSTOMER  
PROVIDER  
MECHANIC  
OPERATIONS_ADMIN  
VERIFICATION_ADMIN  
FINANCE_ADMIN  
SUPER_ADMIN
```

9. Core Data Model

```
users  
vehicles  
providers  
provider_documents  
provider_services  
provider_availability  
services  
bookings  
booking_locations  
booking_status_history  
provider_location_updates  
job_cards  
inspections  
inspection_items  
estimates  
estimate_items
```

invoices
invoice_items
payments
payouts
reviews
notifications
messages
support_tickets
audit_logs

Use UUIDs for externally exposed identifiers. Store created_at/updated_at timestamps consistently. Add soft-delete only where business/audit requirements justify it.

10. Geospatial Data

PostGIS should store provider and booking coordinates. Spatial indexes should support nearby-provider queries.

Example query intent:
Find ACTIVE providers
within service radius
that support requested service
and vehicle type
ordered by distance / ETA / score.

- Use geography/geometry types consistently
- Create GiST spatial indexes
- Never trust client-supplied distance
- Calculate distance server-side
- Restrict active-job location access
- Avoid indefinite raw GPS retention

11. Booking State Machine

REQUESTED
→ SEARCHING
→ PROVIDER_ASSIGNED
→ ACCEPTED
→ ON_THE_WAY
→ ARRIVED
→ INSPECTION
→ ESTIMATE_PENDING
→ APPROVED
→ IN_PROGRESS
→ COMPLETED
→ INVOICE_GENERATED
→ PAYMENT_COMPLETED
→ CLOSED

Terminal alternatives:
CANCELLED
EXPIRED
DISPUTED

State transitions must be validated server-side. Each transition should create a booking_status_history record.

12. Provider Matching

Matching should be implemented as a deterministic, testable service for MVP.

1. Query available providers in geographic radius
2. Filter by requested service
3. Filter by vehicle capability

4. Filter by provider availability
5. Calculate distance / ETA
6. Apply rating and performance score
7. Rank candidates
8. Dispatch request
9. Apply response timeout
10. Retry next eligible candidate if needed
11. Lock accepted assignment transactionally

The matching algorithm should be versioned so future scoring changes can be evaluated without rewriting historical jobs.

13. Real-Time Architecture

```

PROVIDER DEVICE
↓ GPS
Location API / WebSocket
↓
Redis presence / short-lived state
↓
RoadResQ backend
↓ WebSocket events
CUSTOMER DEVICE
  
```

Booking events:
 provider.accepted
 provider.on_the_way
 provider.arrived
 job.started
 estimate.created
 estimate.approved
 job.completed
 payment.updated

- Authenticate WebSocket connections
- Authorize channel membership
- Throttle GPS updates
- Drop stale location data
- Use server timestamps
- Reconnect clients safely
- Persist important business events in PostgreSQL

14. Location Update Requirements

- Provider sends location only while online/active according to product rules
- Server validates coordinates and authenticated provider identity
- Location update frequency should balance ETA quality and battery/data use
- Customer receives only the active provider location relevant to their booking
- Historical raw GPS points should have a defined retention policy

15. REST API Design

```

POST /v1/auth/request-otp
POST /v1/auth/verify-otp
GET /v1/me
GET /v1/vehicles
POST /v1/vehicles
GET /v1/services
POST /v1/bookings
GET /v1/bookings/{id}
POST /v1/bookings/{id}/cancel
  
```

POST /v1/bookings/{id}/approve-estimate
GET /v1/providers/nearby
POST /v1/providers/{id}/availability
POST /v1/provider/requests/{id}/accept
POST /v1/provider/requests/{id}/decline
PATCH /v1/jobs/{id}/status
POST /v1/jobs/{id}/inspection
POST /v1/jobs/{id}/estimate
POST /v1/jobs/{id}/complete
GET /v1/invoices/{id}
POST /v1/payments
POST /v1/reviews

All write endpoints must validate ownership/role and return stable machine-readable error codes.

16. API Standards

- Version APIs under /v1
- JSON request/response format
- ISO-8601 timestamps in UTC
- Consistent pagination
- Idempotency keys for payment/booking-sensitive writes
- Correlation/request ID on every request
- Structured validation errors
- OpenAPI documentation generated from FastAPI

17. Payment Architecture

Customer
↓
Create payment intent/order
↓
Payment Gateway
↓
Webhook
↓
Verify signature
↓
Transactional payment update
↓
Invoice / booking state update

- Never trust client payment-success callbacks
- Verify gateway webhooks server-side
- Use idempotency for retries
- Store gateway transaction identifiers
- Separate payment state from booking state
- Maintain immutable financial records

18. Notifications

- Push notifications for mobile
- Web notifications where appropriate
- SMS/OTP provider for authentication and critical fallback
- Email for invoices/support when useful
- Notification preferences per user
- Persist notification delivery status

Notification dispatch should use background jobs so API requests do not block on external providers.

19. Background Workers

- Provider request timeout processing
- Matching retries
- Notification delivery
- Invoice generation
- Payment reconciliation
- Provider payout calculations
- Analytics aggregation
- Scheduled maintenance reminders
- Cleanup/retention jobs

20. Caching Strategy

- Redis for provider online/presence state
- Cache static service catalog
- Cache frequently accessed provider summary data cautiously
- Never use cache as the sole source of truth for bookings or payments
- Use explicit TTLs
- Invalidate cache after important profile/service changes

21. File & Document Storage

- Object storage for provider documents, vehicle images, inspection photos, and invoice files
- Generate signed temporary URLs
- Validate file type and size
- Virus/malware scanning where available
- Store metadata in PostgreSQL
- Do not expose private storage buckets directly
- Use lifecycle rules for expired temporary assets

22. Security Architecture

- TLS for all network traffic
- Strong password hashing if passwords are used
- JWT/session security
- RBAC + object-level authorization
- Input validation
- SQL parameterization/ORM protections
- CSRF protection where cookie-based auth is used
- Rate limiting
- Secure headers
- Secrets stored outside source control
- Audit logs for privileged actions

23. Privacy & Data Protection

RoadResQ handles location, vehicle, contact, payment, provider-document, and service-history data. The system should apply data minimization, purpose limitation, controlled access, defined retention periods, and deletion/export workflows where legally applicable.

- Encrypt sensitive data in transit and at rest where supported
- Limit location visibility to active operational needs
- Log access to sensitive administrative data
- Define retention policies before production
- Do not store unnecessary payment card data

24. Observability

- Structured JSON application logs
- Request correlation IDs
- Error tracking
- Metrics for API latency and error rate
- Booking lifecycle metrics
- Matching acceptance/timeout metrics
- WebSocket connection metrics
- Payment webhook monitoring
- Database performance monitoring
- Infrastructure health checks

25. Key Technical Metrics

- p50/p95/p99 API latency
- 5xx error rate
- Provider matching latency
- Provider acceptance rate
- Booking state transition failures
- WebSocket disconnect/reconnect rate
- Location update freshness
- Payment webhook success rate
- Queue processing latency
- Database connection utilization

26. Testing Strategy

- Unit tests for domain rules and matching
- Integration tests for PostgreSQL/PostGIS
- API contract tests
- State-machine transition tests
- Payment webhook tests
- Authorization tests
- WebSocket tests
- End-to-end customer emergency flow
- Load tests for matching and active-booking traffic
- Security testing before production

27. Critical E2E Test

Customer login
→ Add vehicle
→ Create emergency booking
→ Confirm location
→ Matching finds provider
→ Provider accepts
→ Customer receives event
→ Provider sends location
→ Customer sees tracking
→ Provider marks arrived
→ OTP verified
→ Inspection created
→ Estimate submitted
→ Customer approves
→ Job completed
→ Invoice generated
→ Payment confirmed
→ Review submitted

28. Error Handling

- No provider available → show retry / expanded search option
- Provider timeout → dispatch next candidate
- Provider cancellation → rematch where safe
- GPS unavailable → request manual location
- Payment failure → keep job/payment states consistent and allow retry
- Duplicate request → idempotency protection
- Network loss → client reconnect and re-fetch authoritative state
- Invalid state transition → reject without partial mutation

29. Concurrency & Transaction Rules

- Use database transactions for assignment, payment state, and critical state transitions
- Prevent two providers from claiming the same assignment
- Use row locks or equivalent concurrency controls where required
- Use unique constraints for idempotency keys and external transaction IDs
- Treat PostgreSQL as the source of truth for transactional state

30. Deployment Architecture

INTERNET
↓
CDN / WAF
↓
Load Balancer
↓
Next.js / API Instances
↓
FastAPI Application
↙ ↘
PostgreSQL Redis
+ PostGIS ↓
↘ Workers
↘ ↓
External Integrations
Maps • Payments • SMS • Push

Containerize services with Docker. Use separate environments for development, staging, and production.

31. Environment Configuration

- Separate environment variables by environment
- Never commit secrets
- Use secret management in production
- Maintain example environment files without credentials
- Validate required configuration at startup
- Rotate credentials periodically

32. CI/CD

- Lint and format checks
- Type checking
- Unit/integration tests
- Build frontend/backend containers
- Security/dependency scanning
- Database migration validation
- Deploy to staging
- Smoke tests
- Promote approved build to production

33. Database Migration Strategy

- Use versioned migrations
- Review destructive migrations manually
- Back up production before high-risk migrations
- Prefer additive schema changes for zero/low downtime
- Add indexes carefully on large tables
- Test migration rollback/recovery procedures

34. Scalability Plan

Start with a modular monolith. Scale stateless API instances horizontally. Move heavy or independent workloads to workers. Introduce service separation only when measurable bottlenecks or team boundaries justify it.

MVP

Single API + Worker + PostgreSQL + Redis

Growth

Multiple API replicas

Read replicas where needed

Dedicated worker pools

CDN/object storage

Geospatial query optimization

Large scale

Domain services

Event bus

Partitioning / archival

Dedicated real-time infrastructure

35. Availability & Reliability

- Health endpoints for application and dependencies

- Database backups
- Point-in-time recovery where supported
- Graceful shutdown
- Retry external calls with bounded backoff
- Circuit breakers for unreliable external services
- Idempotent background jobs
- Recovery procedures documented and tested

36. Performance Requirements

- API target: p95 under 500 ms for normal non-geospatial reads under expected MVP load
- Emergency request creation should remain responsive under normal conditions
- Nearby-provider query should use spatial indexes
- WebSocket event delivery should be near real-time under normal network conditions
- Heavy work such as notifications and document processing must not block API requests

37. Admin Security

- Separate admin authentication path or elevated assurance
- MFA recommended/required for privileged roles
- Least-privilege permissions
- Audit all provider verification, refunds, suspensions, and manual booking changes
- Restrict bulk export of personal/location data
- Session timeout and revocation

38. Audit Logging

Audit logs should capture important privileged and financial actions.

```
audit_logs
- id
- actor_user_id
- action
- resource_type
- resource_id
- metadata
- ip_address (if appropriate)
- user_agent (if appropriate)
- created_at
```

39. Third-Party Integrations

- Maps/geocoding/routing provider
- Payment gateway
- OTP/SMS provider
- Push notification provider
- Email provider
- Object storage
- Error monitoring
- Analytics

Integrations must be isolated behind adapter modules so providers can be replaced without rewriting domain logic.

40. AI Extension Architecture

AI should be an optional domain service rather than a dependency of the core emergency flow.

```
Customer symptoms / media
↓
AI service
↓
Structured assistance suggestion
↓
RoadResQ validation
↓
Service category / provider matching
```

AI output must be treated as advisory and should not override safety-critical rules or provider qualifications.

41. Better Garage Integration

RoadResQ should expose shared domain primitives that can later support Better Garage: users, vehicles, customers, providers, jobs, service history, invoices, payments, and staff.

Avoid coupling RoadResQ booking logic directly to garage ERP workflows. Use shared IDs/events or a well-defined integration boundary.

42. Recommended Repository Structure

```
roadresq/
├── apps/
│   ├── web/
│   ├── mobile/
│   └── api/
├── packages/
│   ├── ui/
│   ├── types/
│   └── config/
├── infra/
│   ├── docker/
│   ├── nginx/
│   └── migrations/
├── docs/
└── tests/
```

43. MVP Engineering Priority

- P0: Auth, users, vehicles, providers
- P0: Services and emergency bookings
- P0: Matching and assignment locking
- P0: Booking state machine
- P0: Provider accept/reject
- P0: Basic geolocation
- P0: Admin provider verification
- P1: WebSocket status updates
- P1: Job cards and estimates
- P1: Invoices and payments
- P1: Reviews
- P2: Advanced live tracking, chat, analytics, AI

44. Technical Risks

- Low provider density in a location

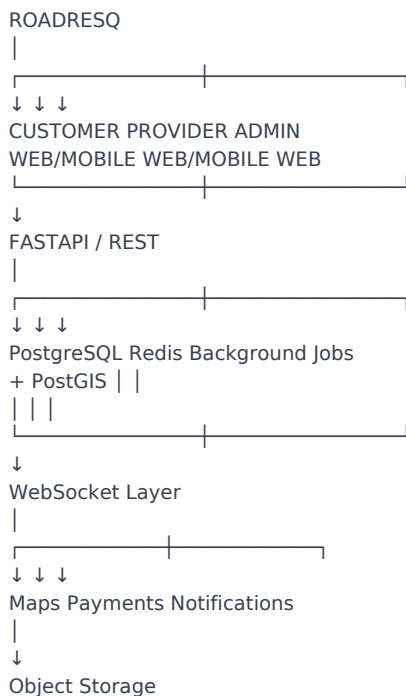
- Inaccurate GPS or poor connectivity
- Race conditions during provider assignment
- Payment webhook delays or duplicates
- Third-party maps/payment outages
- Location privacy concerns
- Fraudulent provider/customer activity
- High real-time traffic during growth

Each risk should have monitoring, fallback behaviour, and an operational owner before production.

45. Definition of Done — MVP

- All P0 flows implemented and tested
- Role-based access verified
- Booking state transitions are server-controlled
- Provider assignment is concurrency-safe
- Emergency request works on mobile networks
- Basic location display works
- Payment state is verified server-side
- Audit logs cover privileged actions
- Production secrets are externalized
- Database backups configured
- CI/CD pipeline passes tests
- Monitoring and error reporting are active

46. Final Technical Architecture



47. Technical Conclusion

RoadResQ should begin as a secure, modular, geospatially aware application rather than an unnecessarily distributed microservice system. PostgreSQL/PostGIS remains the transactional source of truth, Redis handles short-lived real-time/cache workloads, WebSockets deliver active-job events, and background

workers isolate slow external integrations.

The architecture deliberately keeps the customer assistance loop independent from future garage-management functionality, allowing RoadResQ to grow into a larger automotive ecosystem and integrate cleanly with Better Garage.